

00_programme_master_plan

HelixLLM Full-Extension Programme

— Master Plan & Research Index

Document	Master programme plan + research index
Path	docs/research/ 07.2026/00_master/ 00_programme_master_plan.md
Revision	1
Created	2026-07-06
Status	RESEARCH IN PROGRESS (Phase R)
Constitution HEAD followed	0882b9e (through §11.4.182)
Track/branch	(T1/main)
Release prefix (§11.4.151)	resolve from HELIX_RELEASE_PREFIX else helix_code

Anti-bluff notice (§11.4 / §11.4.123): every number, model recommendation, and capability claim in this programme’s documents MUST carry a cited source or a captured runtime proof. Design estimates are labelled as such; measured results replace them. UNCONFIRMED: / PENDING_FORENSICS: mark anything not yet proven (§11.4.6). No placeholder or mocked implementation ships (§11.4.2 / CONST-050).

1. Objective (verbatim intent, decomposed)

Fully extend **HelixLLM** into a production-grade local-first model platform running on the current host (**Gigabyte RTX 5090 AORUS MASTER, 32 GB VRAM, Blackwell; Linux; strong CPU/RAM**), expose it **through HelixAgent** to **HelixCode** and any **CLI agent** configured for HelixAgent, and drive it to **near-frontier coding + agentic quality** while serving **multiple concurrent instances × multiple subagents** (target 6–12 concurrent sequences).

Everything runs **locally, in rootless-podman containers via the containers submodule** (§11.4.76 / §11.4.161), is **fully tested** (unit/integration/e2e/security/stress/chaos + Challenges + HelixQA autonomous sessions), produces **rock-solid captured proofs** (§11.4.5 / §11.4.69 / §11.4.107 / §11.4.123), and is **kept running for live operator testing** when complete.

2. Capability scope (the complete, non-negotiable list)

Nothing in this list may be forgotten, skipped, or stubbed (§11.4.122 / §11.4.124 / CONST-050).

2.A Core local serving

- Local llama.cpp (or better infra — to be decided by research stream 01) on the RTX 5090.
- Primary coding + agent models (baseline hypothesis, to be verified by stream 01): Qwen2.5-Coder-32B-Instruct (heavy coding), Qwen2.5-32B-Instruct (CLI agents / one-model-both), DeepSeek-Coder-V2-Lite (fast), plus 2026 successors (Qwen3-Coder, DeepSeek-V3/R1 distills, GLM-4.x, Devstral...).
- Multi-instance / multi-agent concurrency: continuous batching, --parallel, KV-cache quant (q8_0), flash attention; context-cap + VRAM budgeting for 6–12 concurrent sequences.

2.B Exposure & standards

- **HelixAgent exposes HelixLLM + the AI ensemble** + any other exposed models.
- **Both OpenAI-compatible and Anthropic-compatible REST endpoints** for all our APIs.
- Claude Code / HelixCode auto-recognize the exposed models immediately (standard model-list surface).
- MCP + ACP integration; MCP + “OKF” (Google OKF — clarify/verify in stream 05); function/tool-calling standards.

2.C Extended capabilities (each fully local model + full provider coverage)

- **Vision**: local VLM (image understanding, screenshots, OCR-in-image), exposed via OpenAI vision endpoint.
- **Generative**: image generation, **video (WAN + LTX mandatory)**, vector drawing, illustration, sketch/design, **vectorizing pixel graphics**.

- **Translation:** professional-grade local translation model(s) + all major translation providers.
- **Embeddings + RAG + HelixMemory** heavy use (durable agent memory).
- **STT = Whisper + OCR = Tesseract** wired everywhere they add capability, incl. HelixQA vision testing.
- **codegraph + opendesign** as core under-the-hood providers across HelixCode/HelixAgent/HelixLLM/LLMsVerifier (§11.4.78–80 codegraph; §11.4.162 opendesign).

2.D Provider coverage (LLMsVerifier + HelixAgent) — research stream 06

Latest models for all existing providers, **plus** every not-yet-supported provider/model, including explicitly: **Poe, Perplexity, Sakana Fugu (+ family), Qwythos 9B, Google OKE, GOT family + GPT SOL, Subquadratic (+ models), Xiaomi, Tencent (+ Yuanbao), WAN + LTX video, major generative providers, major vision providers/models, major translation providers/models.** (Each name to be verified for exact API/endpoint/auth/model-list; unverifiable ones flagged, never faked — §11.4.6.)

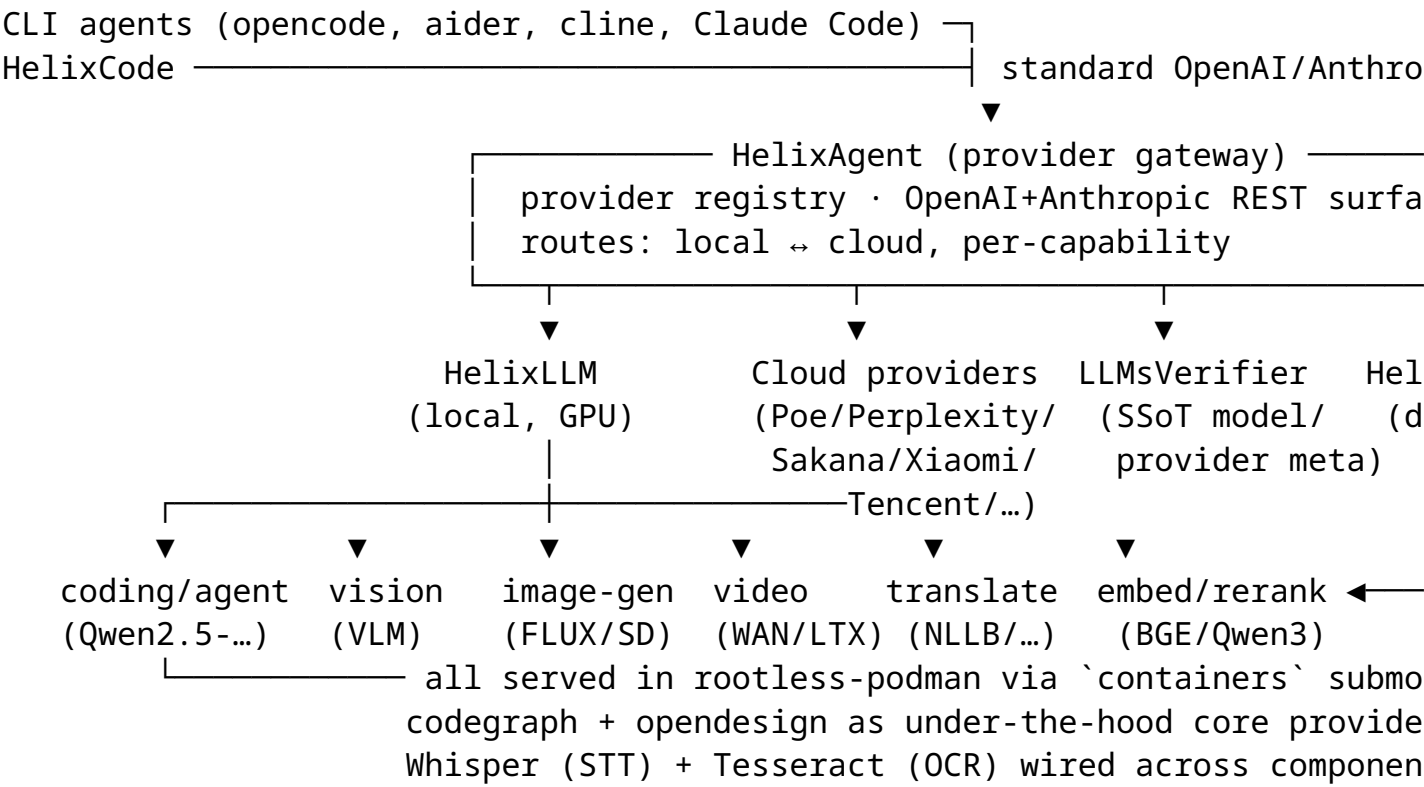
2.E Tooling / integration

- **LLMsVerifier extended** to work flawlessly with HelixLLM (verify local + all new providers).
- **claude_toolkit** (/home/milos/Factory/projects/tools_and_research/claude_toolkit) recognizes **HelixAgent as a provider when on PATH**, creates provider aliases for HelixAgent + all its exposed models (AI ensemble, HelixLLM, others), with extended docs/guides/diagrams; then updates its vendored LLMsVerifier to latest and fully incorporates it.
- **Setup scripts** install HelixCode + all power sub-systems and add ALL to PATH: HelixCode, HelixAgent, HelixLLM, LLMsVerifier, + others.
- **Comprehensive provider-verification test** for the HelixAgent alias, like every other provider alias.

2.F Quality gate

- Full test-type coverage (CONST-050 / §11.4.4(b)) + Challenges (challenges) + HelixQA autonomous sessions (extended for the new vision capabilities). Rock-solid captured proofs; zero bluff.
- All documentation everywhere re-evaluated + extended; new docs, user guides, manuals, diagrams, schemes, SQL/templates, illustrations (§11.4.153 four-format incl DOCX where that class applies).

3. Target architecture (working hypothesis — to be validated)



4. Research streams (Phase R) — status board

#	Stream	Output doc	Status
00	Codebase inventory (grounding)	00_master/ 00_codebase_inventory.md	DONE
01	Local models + serving infra (RTX 5090, concurrency)	01_local_models_serving/...	• running
02	Vision + generative (image/video WAN+LTX/ vector)	02_vision_generative/...	• running •
03	Translation (local + cloud)	03_translation/...	running
04		04_embeddings_rag/...	running

#	Stream	Output doc	Status
	Embeddings + RAG + HelixMemory		
05	MCP + ACP + OKF + tool- calling standards	05_mcp_acp_protocols/...	• running
06	Provider coverage (Poe/ Perplexity/ Sakana/Xiaomi/ Tencent/...)	06_providers_coverage/...	• running •
07	Whisper (STT) + Tesseract (OCR) integration	07_stt_ocr_whisper_tesseract/...	• running
08	codegraph + opendesign as core providers	08_codegraph_opendesign/...	running
09	Containers/ rootless-podman deployment patterns	09_containers_deployment/...	• running •
10	LLMsVerifier ⇌ HelixLLM extension design	10_llmsverifier_helixagent/...	running
11	claude_toolkit provider recognition + aliasing	11_claude_toolkit/...	🟢 running
12	HelixQA extension for vision + full test matrix	12_helixqa_testing/...	🟢 DONE
99	Multi-pass risk / danger-zone / bottleneck analysis	99_risk_analysis/ 99_risk_bottleneck_analysis.md	DONE

PHASE R COMPLETE (2026-07-06). 13 cited reports + 02_cross_cutting_foundations_ADR.md + 03_open_clarifications.md (all 4 operator-resolved) + 99_risk_bottleneck_analysis.md. **PHASE P COMPLETE:** 04_implementation_plan.md (10 phases → tasks → subtasks, critical-path sequenced). ► **NEXT: Phase 0** (host GPU foundation — the unblocking prefix).

Inventory (00) key facts (full detail in 00_codebase_inventory.md): HelixLLM is a real Go submodule (github.com/HelixDevelopment/HelixLLM, at submodules/helix_llm/ + a second checkout under dependencies/HelixDevelopment/helix_llm/) already serving OpenAI + Anthropic APIs, RAG, a ReAct agent, and a scored provider fallback with a real internal/brain/llamacpp.go HTTP client to a llama.cpp server; HelixAgent has ~45 providers + its own OpenAI/Anthropic/Google REST server. → This is an **extend**, not a greenfield build. Grounded gaps: (1) no RTX-5090 GPU build/launch wiring for llama.cpp; (2) no vision/translation serving surface; (3) HelixLLM not pinned as a root submodule (two checkouts → drift risk); (4) LLMsVerifier missing CONST-040 capability probes; (5) host-wiring rot (.mcp.json macOS paths, opendesign disabled, claude_toolkit keys off env-var not PATH).

Waves 2–3 (streams 05–12) dispatch after stream 00 (inventory) returns, so provider/integration research is grounded in the actual HelixAgent/LLMsVerifier/claude_toolkit interfaces (§11.4.6).

5. Phase plan (high-level — refined into fine-grained tasks/subtasks after Phase R)

- **Phase R — Research & inventory** (current): streams 00–12 + risk 99 → cited reports.
- **Phase P — Planning:** synthesize research into a full implementation plan with phases → tasks → subtasks down to file/function/line level; multi-pass danger-zone analysis (§11.4.92 5-pass); ADRs.
- **Phase 1 — Local serving core:** HelixLLM llama.cpp/infra service in rootless podman; primary coding/agent model live; OpenAI+Anthropic endpoints; HelixAgent provider for HelixLLM.
- **Phase 2 — LLMsVerifier + HelixAgent provider gateway:** verify local + new providers; model-list surface.
- **Phase 3 — Extended capabilities:** vision, generative (image/video WAN+LTX/vector), translation, embeddings/RAG/HelixMemory, Whisper/Tesseract — each a local model + provider coverage.
- **Phase 4 — Integrations:** codegraph + opendesign core wiring; MCP/ACP/OKF.
- **Phase 5 — claude_toolkit:** HelixAgent provider recognition + aliases + docs; update vendored LLMsVerifier.

- **Phase 6 — Setup + PATH:** install scripts for the whole stack onto PATH.
- **Phase 7 — Test/Challenges/HelixQA:** full matrix + autonomous QA sessions + captured proofs.
- **Phase 8 — Live run:** keep the system running for operator testing; resumption doc (§11.4.131).

Each implementation phase is subagent-driven (§11.4.70), reviewed independently (§11.4.125 / §11.4.142), multi-angle-researched per change (§11.4.145), reproduce-first tested (§11.4.146), and gated by runtime-signature verification on a clean deploy (§11.4.108).

6. Binding governance anchors (non-exhaustive)

Anti-bluff family (§11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.123); rootless containers §11.4.161 + containers submodule §11.4.76; opendesign §11.4.162; codegraph §11.4.78–80; deep multi-angle research per change §11.4.150 + latest-source §11.4.99; full test-type coverage CONST-050 / §11.4.85 stress+chaos; HelixQA + Challenges §11.4.27 / CONST-050(B); media validation §11.4.163 + window-scoped project-prefixed recordings §11.4.153–155 / §11.4.158–160; no-silent-removal §11.4.122 + investigate-before-remove §11.4.124; release prefix §11.4.151; one canonical branch name §11.4.181; track/branch labels §11.4.182; no-force merge-onto-latest §11.4.113; independent verification §11.4.165.

7. Risk register (seed — expanded by stream 99, multi-pass §11.4.92)

ID	Danger zone / bottleneck	Why it bites	Mitigation direction
RZ-01	Blackwell (sm_120) CUDA support in serving engines	llama.cpp/vLLM may need specific CUDA/build flags; wrong build = no GPU / crashes	Stream 01 verifies exact build flags + CUDA version; capture nvidia-smi + a real inference proof
RZ-02	VRAM exhaustion under 6–12 concurrent agents	KV cache per sequence dominates; naive config OOMs	Context cap + q8_0 KV quant + measured concurrency math (stream 01); hard limits per §11.4.133 target-safety
RZ-03		Aggregate VRAM >> 32	Model residency scheduling / on-

ID	Danger zone / bottleneck	Why it bites	Mitigation direction
	Running MANY services on ONE 32 GB GPU (LLM+VLM+img+video+embed)	GB if all resident	demand load-unload; per-capability separate podman services; single-owner GPU discipline §11.4.119
RZ-04	Unverifiable / mis-named providers (Qwythos 9B, GOT/ GPT SOL, OKF, Fugu)	Names may be ambiguous / not real public APIs	Stream 06 verifies each against latest official sources; flag UNCONFIRMED, never fake (§11.4.6) — ask operator if truly ambiguous
RZ-05	HelixQA vision testing needs Whisper/Tesseract + self-validated analyzers	New capability could pass-bluff (frozen frame, wrong OCR)	§11.4.107/.137/.117/.163 self-validated golden-good/bad analyzers; media-validation pipeline
RZ-06	claude_toolkit is a SIBLING repo (outside monorepo)	Cross-repo change coordination; PATH detection portability	Stream 11 maps the alias mechanism; no hardcoded paths (§11.4.28 / CONST-045)
RZ-07	Large model/media weights not versioned	Fresh clone can't run (§11.4.77)	.gitignore + declared re-obtain mechanism per weight (§11.4.77)
RZ-08	Host safety while driving GPU hard	Thermal/ power; must not harm host or hardware (§11.4.133 / §12)	Thermal-aware limits, captured thermal evidence, no unverified GPU-control writes

8. Next actions (live)

1. Await streams 00–04 (running); on 00 (inventory) return, dispatch waves 2–3 (streams 05–12).

2. Fill §4 status board + per-stream summaries as reports land.
3. After all streams: write the full Phase-P implementation plan (tasks/subtasks to line level) + ADRs.
4. Create session-resumption file (§11.4.131) + docs/CONTINUATION.md update for this programme.
5. Governance cascade follow-up: propagate §11.4.180–182 into helix_code governance files + bump the constitution pointer (CONST-049 step 7) — tracked separately.